

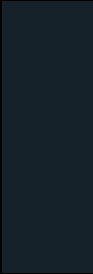


Core Security Fundamentals

Fundamental Web Security Threats & Attacks



Content



Chapter 6: Fundamental Web Security Threats

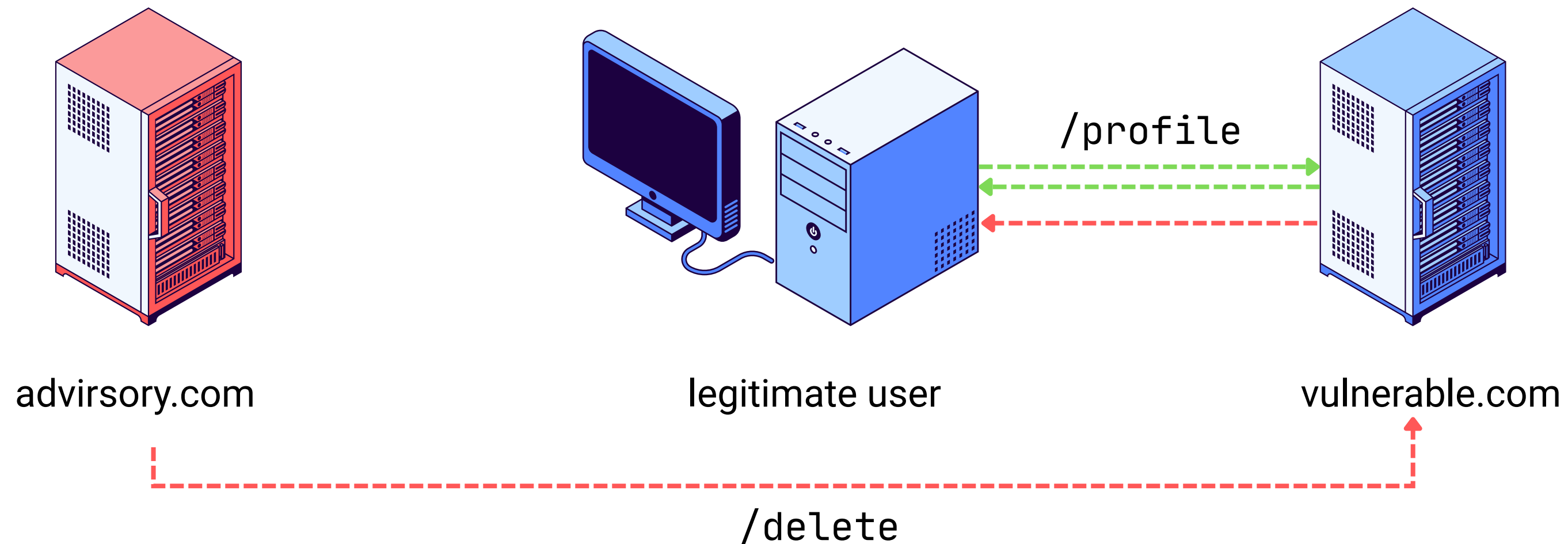


Cross-Site Request Forgery

Cross-site request forgery (CSRF)

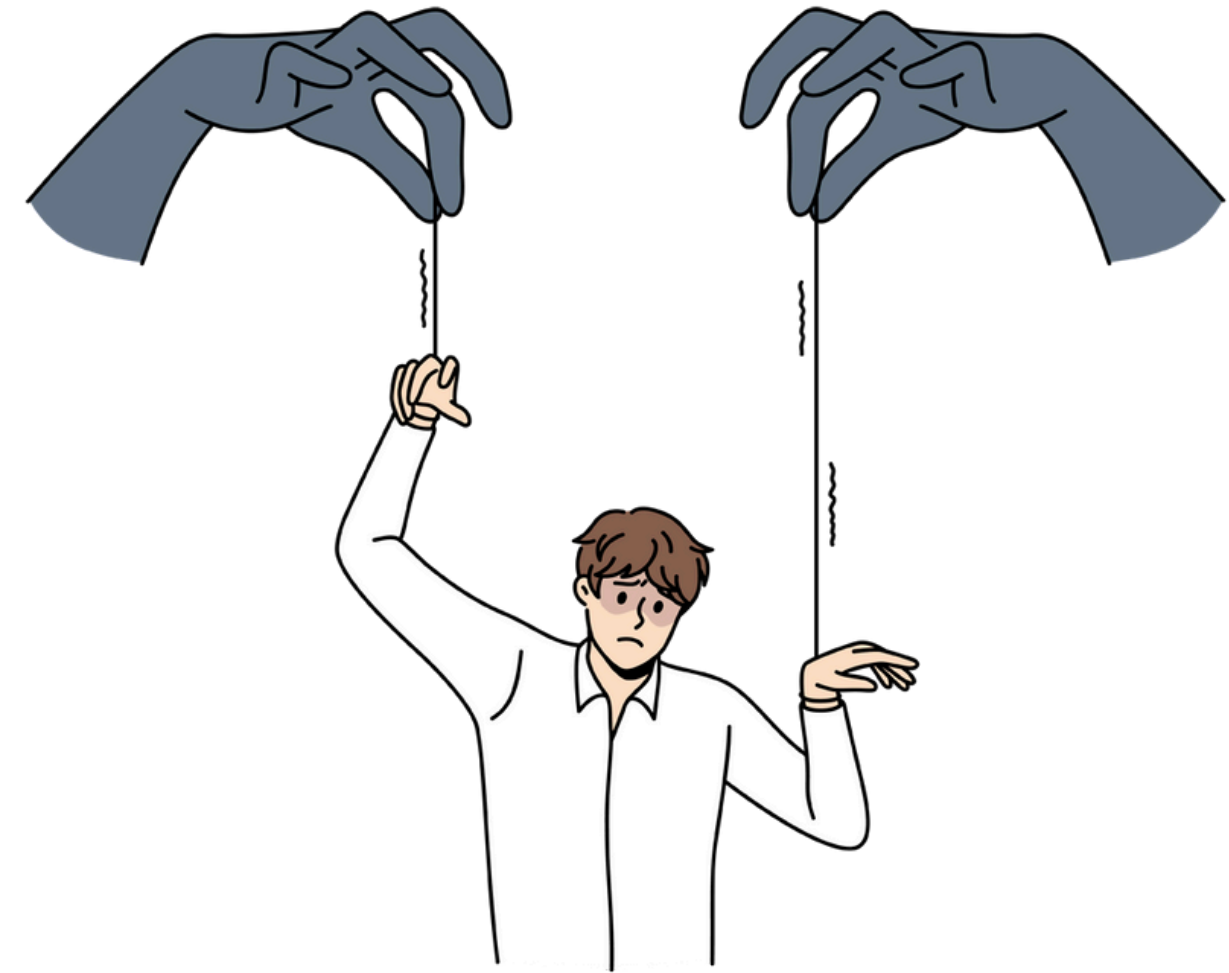
Cross-site request forgery (also known as CSRF) is a web security vulnerability that allows an attacker to induce users to perform actions that they do not intend to perform. It allows an attacker to partly circumvent the same origin policy, which is designed to prevent different websites from interfering with each other.

Web security Academy



What is the impact of a CSRF attack?

In a successful CSRF attack, the attacker causes the victim user to **carry out an action unintentionally**. For example, this might be to change the email address on their account, to change their password, or to make a funds transfer. Depending on the nature of the action, the attacker might be able to gain full control over the user's account. If the compromised user has a privileged role within the application, then the attacker might be able to take full control of all the application's data and functionality.

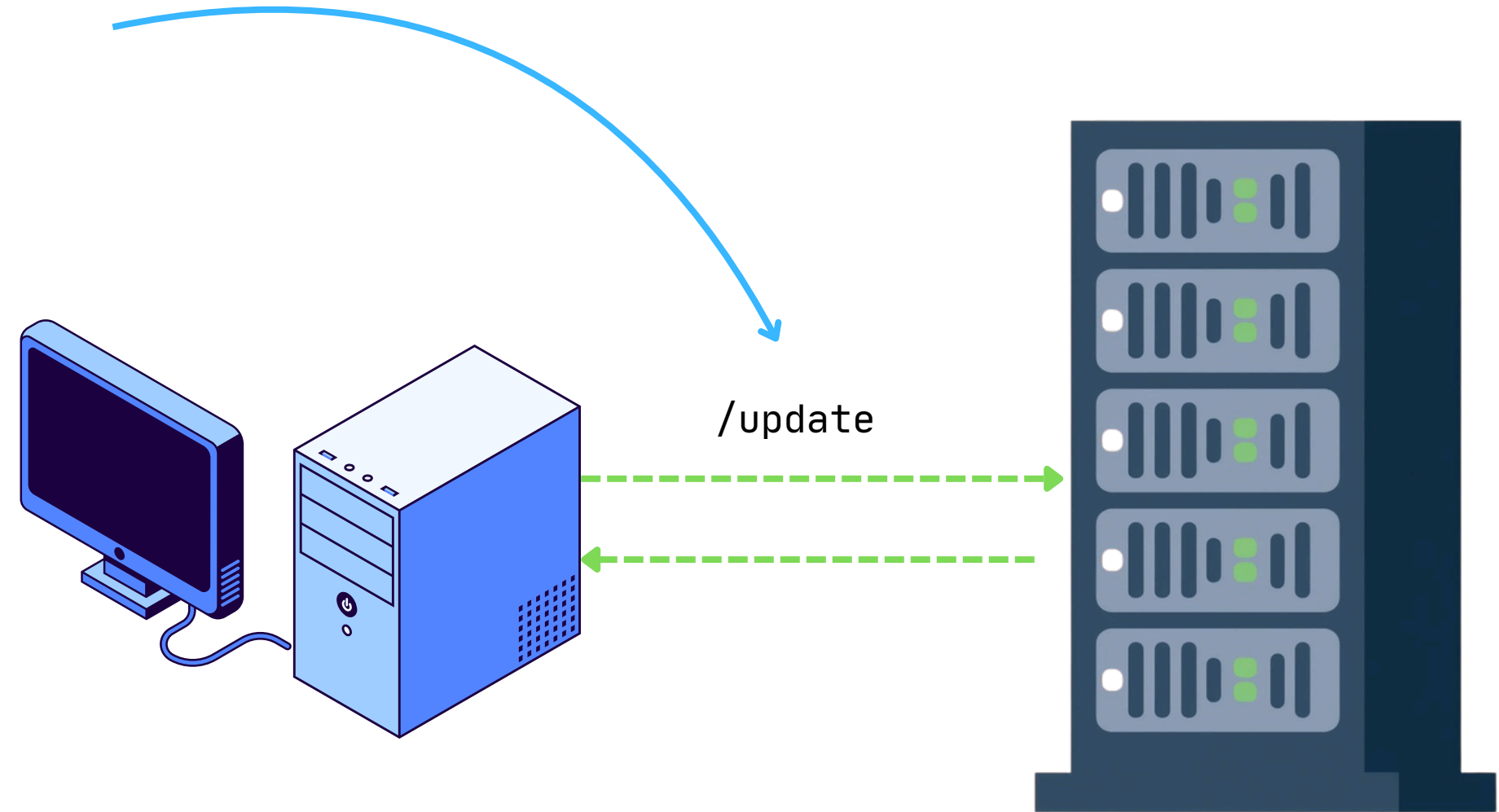




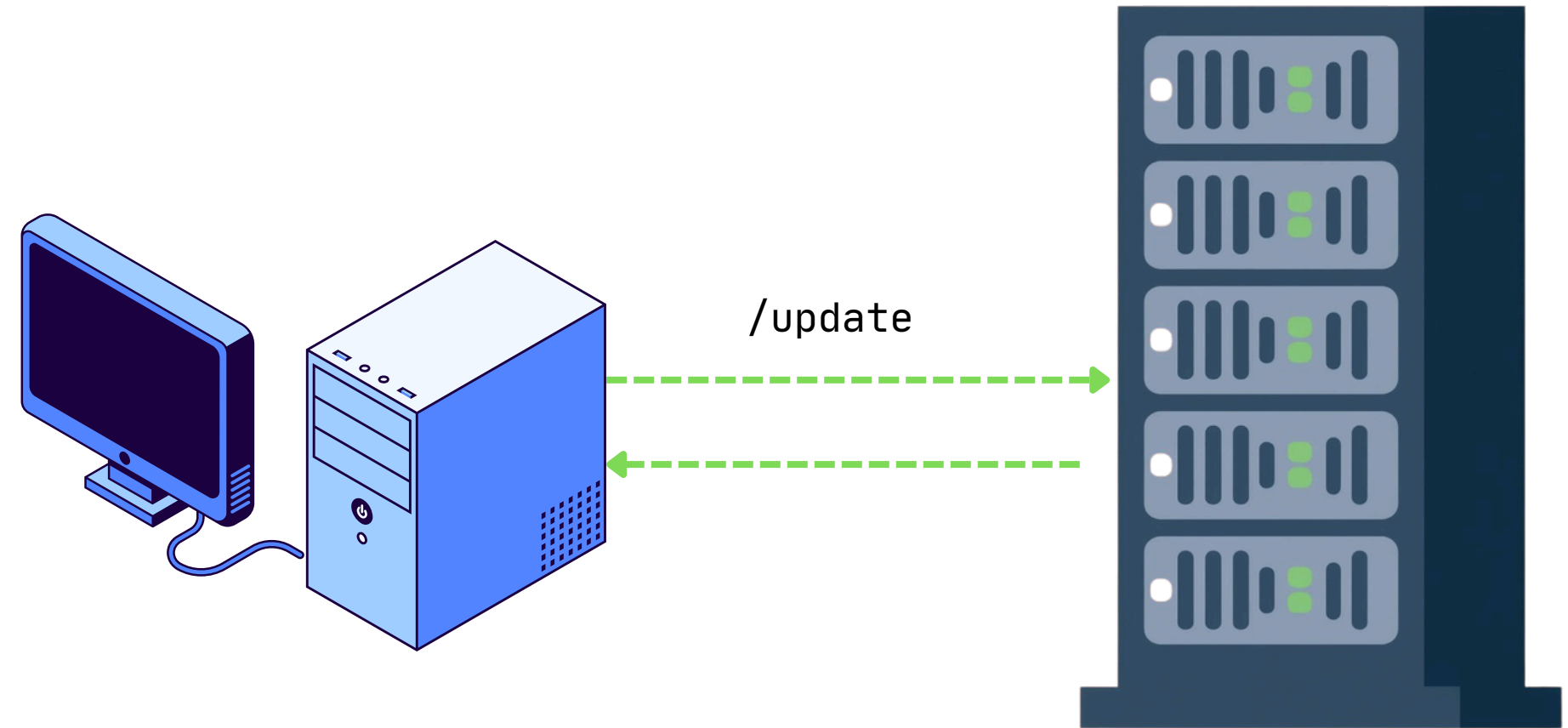
How does CSRF work?

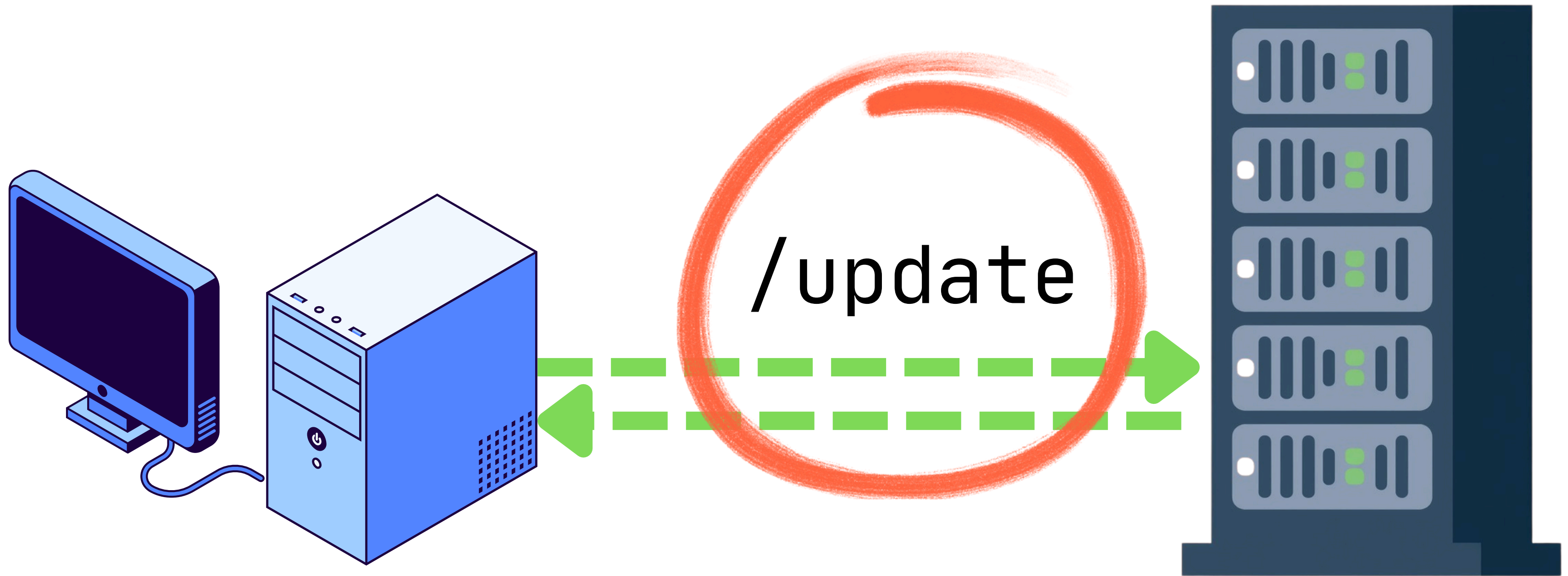


Attacker



`https://vulnerable-website.com/email/update?email=primary@gmail.com`
`Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE`





/update

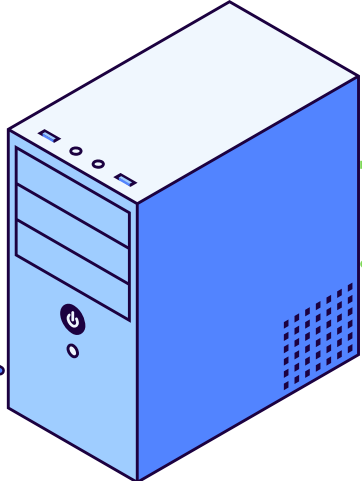
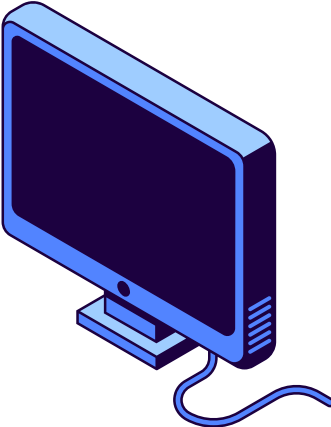
```
POST /email/update HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE
email=primary@gmail.com
```

evil.com

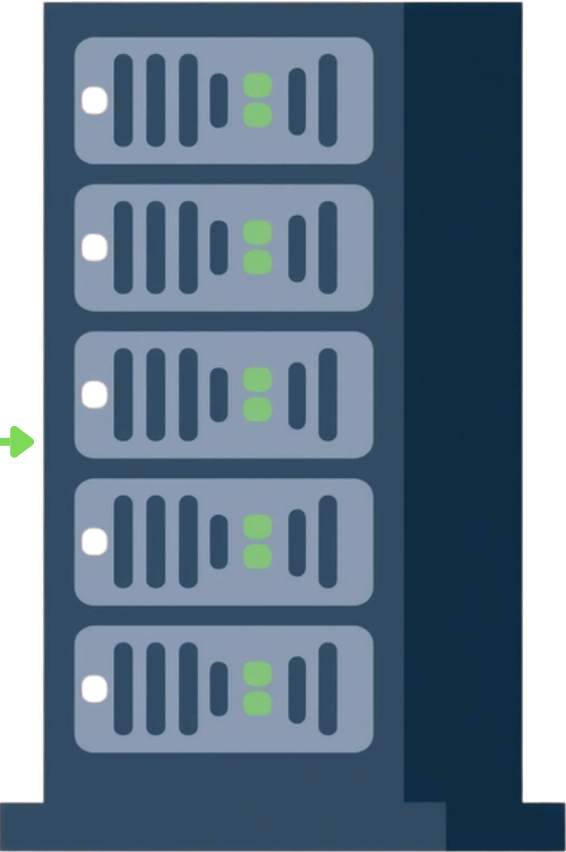


Attacker

```
POST /email/update HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE
email=hacker@gmail.com
```

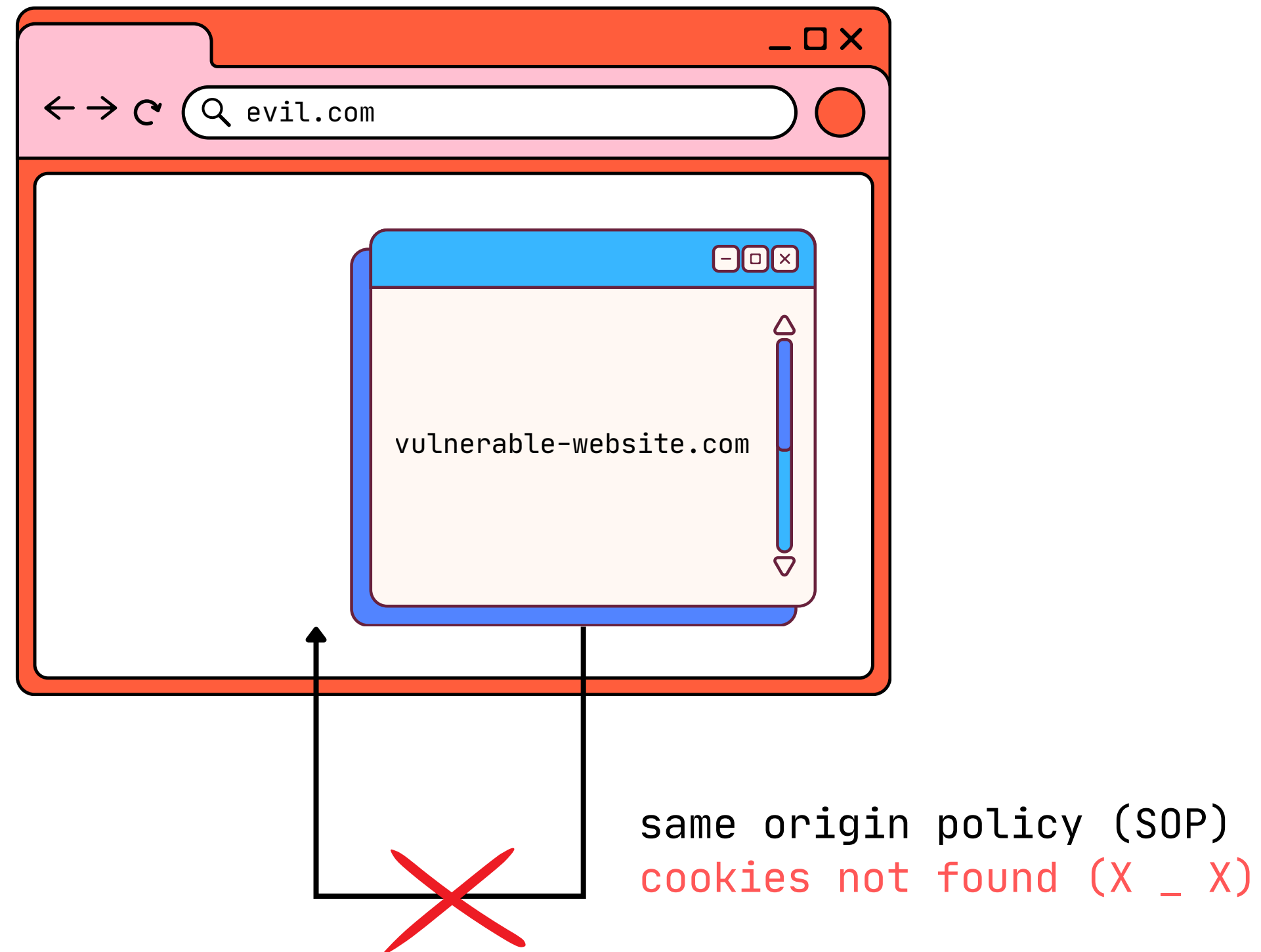


/profile

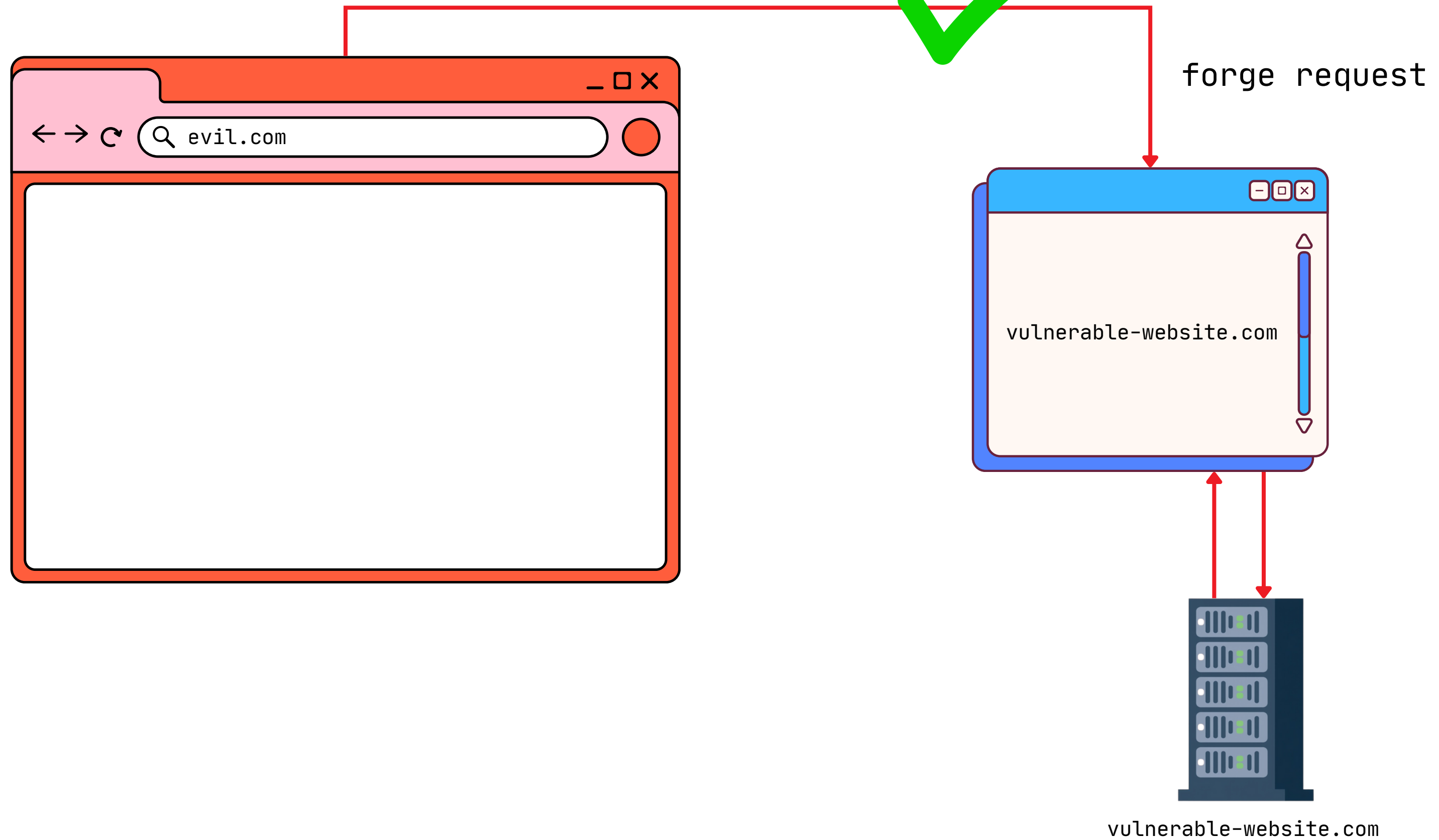


/update

CSRF doesn't violate SOP !



CSRF doesn't violate SOP !

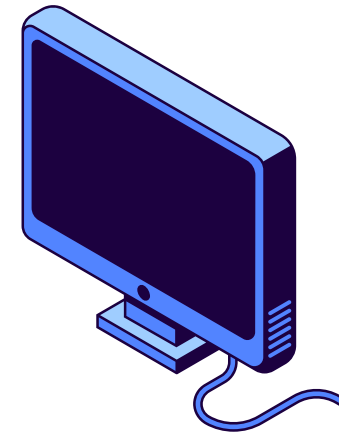


evil.com

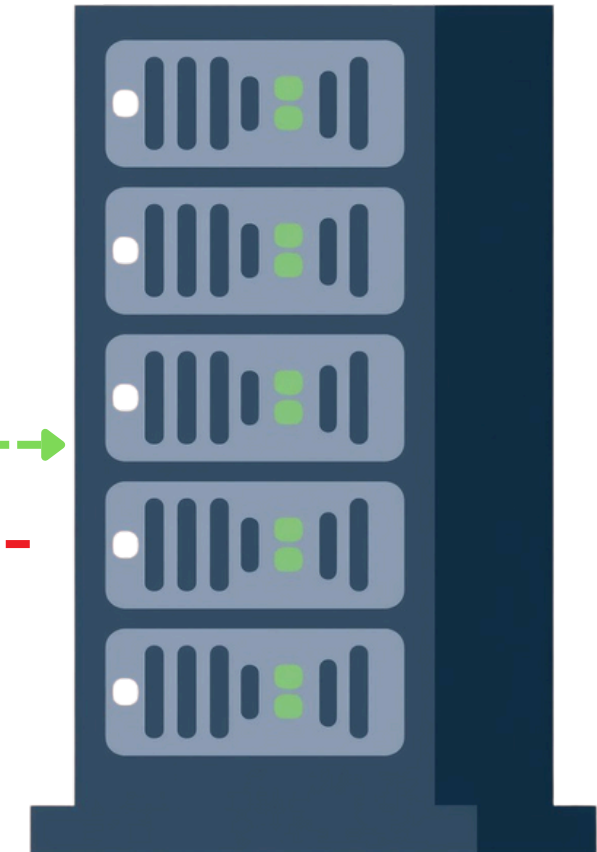


Attacker

```
POST /email/update HTTP/1.1
Host: vulnerable-website.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
Cookie: session=yvthwsztyeQkAPzeQ5gHgTvlyxHfsAfE
email=hacker@gmail.com
```



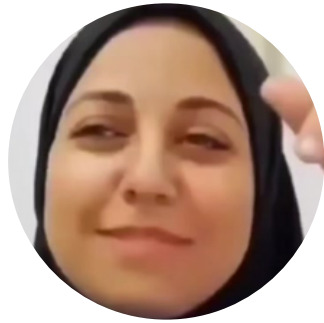
/profile



Compromised !

/update

RitRot.com



views 559k



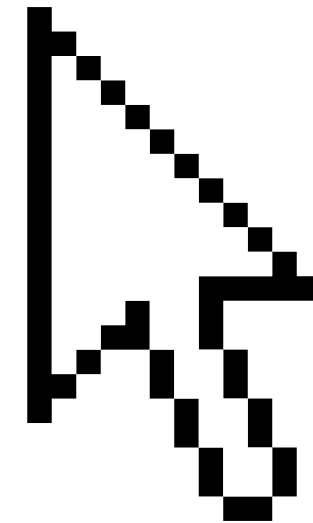
views 120k



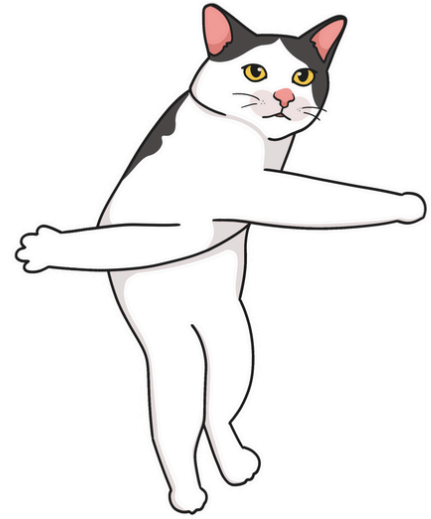
views 410k



views 720k



RitRot.com



stupid video getting
watched by stupid people

Stupid Omar

WoolaWoop
nice Yeah

Yoo! Meow!

Stupid Lisa



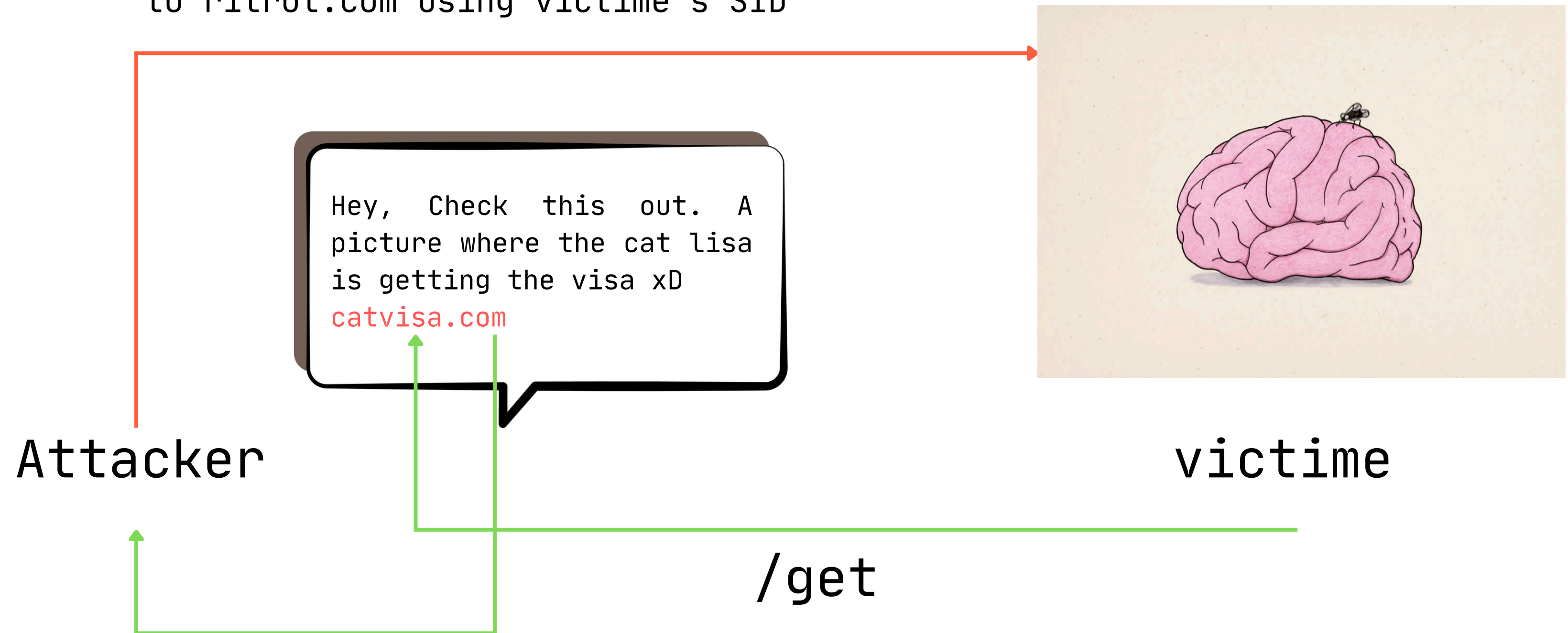
Lisa!Wooah
I'm a dog

Stupid John

Hey, Check this out. A
picture where the cat lisa
is getting the visa xD
catvisa.com

Attacker

`/post /put /delete`
to ritrot.com using victime's SID



Protection against CSRF.



Anti-CSRF Tokens

Nowadays, successfully finding and exploiting CSRF vulnerabilities often involves bypassing anti-CSRF measures deployed by the target website, the victim's browser, or both. The most common defences you'll encounter are as follows:

- CSRF tokens - A CSRF token is a unique, secret, and unpredictable value that is generated by the server-side application and shared with the client. When attempting to perform a sensitive action, such as submitting a form, the client must include the correct CSRF token in the request. This makes it very difficult for an attacker to construct a valid request on behalf of the victim.

Anti-CSRF Tokens

- SameSite cookies - SameSite is a browser security mechanism that determines when a website's cookies are included in requests originating from other websites. As requests to perform sensitive actions typically require an authenticated session cookie, the appropriate SameSite restrictions may prevent an attacker from triggering these actions cross-site. Since 2021, Chrome enforces Lax SameSite restrictions by default. As this is the proposed standard, we expect other major browsers to adopt this behaviour in future.
- Referrer-based validation - Some applications make use of the HTTP Referer header to attempt to defend against CSRF attacks, normally by verifying that the request originated from the application's own domain. This is generally less effective than CSRF token validation.



End. CVE-1803-96. Thank you.